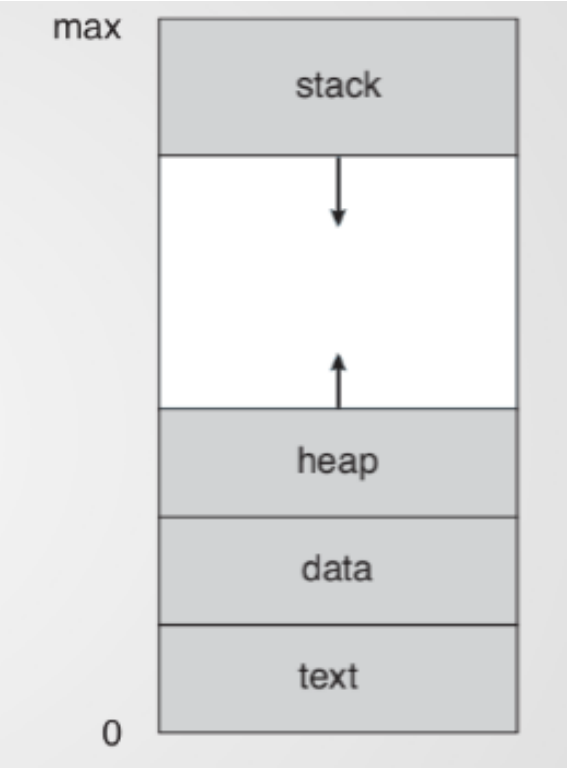
4 - 17 marzo 2026

Processi

Definizione di processo

Il processo è un istanza di un programma in memoria, ogni processo ha il proprio spazio di indirizzamento, questo è composto dalle seguenti cose

- *codice*(text nell'img): questa è l'unica parte in comune tra due processi
- *dati*: strutture statiche che hanno valore sempre (delle variabili valorizzate sin dall'avvio del programma) si trova vicino al codice perché anche questi dati vengono caricati in memoria insieme al codice
- *stack*: una struttura di supporto al processo, in pratica una memoria che gestisce le chiamate a funzione e la gestione delle variabili statiche
- *heap*: memoria dinamica in cui vengono allocate le variabili dinamiche (classi, oggetti, malloc, new)

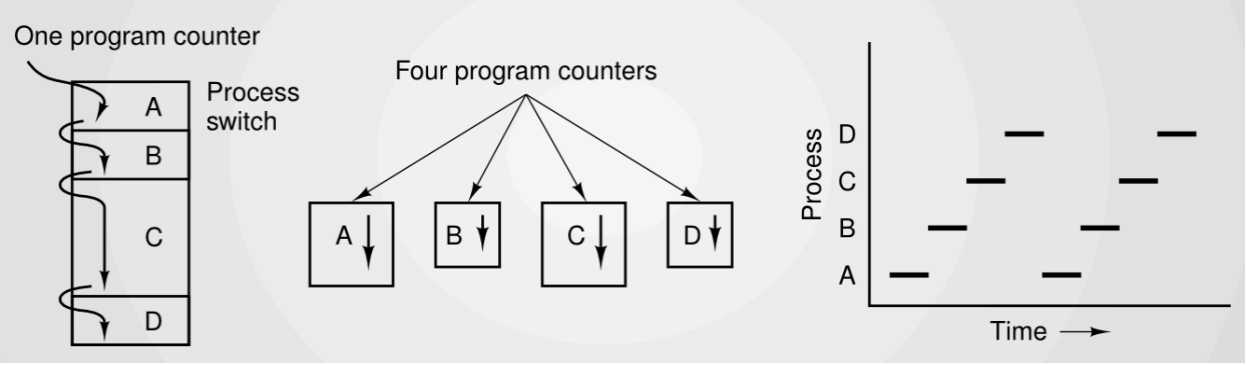


Tip

Usiamo lo heap perché quando si ha un ritorno di una funzione nello stack tutte le variabili allocate vengono eliminate questo non succede con le cose allocate nello heap, che restano fisse fino a quando il programmatore non le elimina

Per ogni processo abbiamo un **Process Control Block** dove sono contenuti diversi metadati, tutti i **PCB** si trovano dentro *una tabella dei processi* (nella realtà non è una tabella ma spesso è un albero rosso nero) ognuno è identificato univocamente. Tra i metadati troviamo diverse informazioni come lo stato del processo, i file aperti, la copia dei valori che deve usare nei registri(quando viene eseguito questi valori vengono caricati nei registri), processi imparentati e allarmi pendenti(interrupt/segnali)

Modelli dei processi



Se si dispone di una sola CPU questa può elaborare un solo processo alla volta, anche se poi nell'uso quotidiano i processi da eseguire sono molti di più, allora la CPU crea delle CPU virtuali usate per dare l'idea che ogni processo abbia la sua CPU personale, questo si chiama *pseudo-parallelismo* perché appunto da l'idea di parallelizzare i processi quando in realtà non lo fa, semplicemente li esegue in maniera ottimizzata saltando da processo a processo in base a dipendenze e altro, questo comunque ne migliora la velocità di esecuzione globale

Creazione e terminazione dei processi

Creazione di un processo processo: Ogni processo ha un PID che è un codice univoco, il processo viene creato in fase di inizializzazione del sistema oppure viene creato un per azione dell'utente, questo viene fatto attraverso 2 metodologie

- **sdoppiamento del padre**:
  - usando fork un comando UNIX che crea un nuovo processo clonando il padre
  - usando exec un comando UNIX che esegue un programma all'interno dello spazio di indirizzamento del processo chiamante, in pratica cancella tutte le informazioni del processo chiamante e inizializza al suo interno il nuovo processo (non crea un nuovo processo, semplicemente libera lo spazio usato da quello chiamante e in quello spazio vuoto esegue il processo nuovo)
- **nuovo processo**: usando CreateProcess, un comando specifico per windows che prende 10 argomenti e crea un nuova entità con PID (Process ID) diverso, uno dei tanti argomenti indica il path del percorso dell'eseguibile

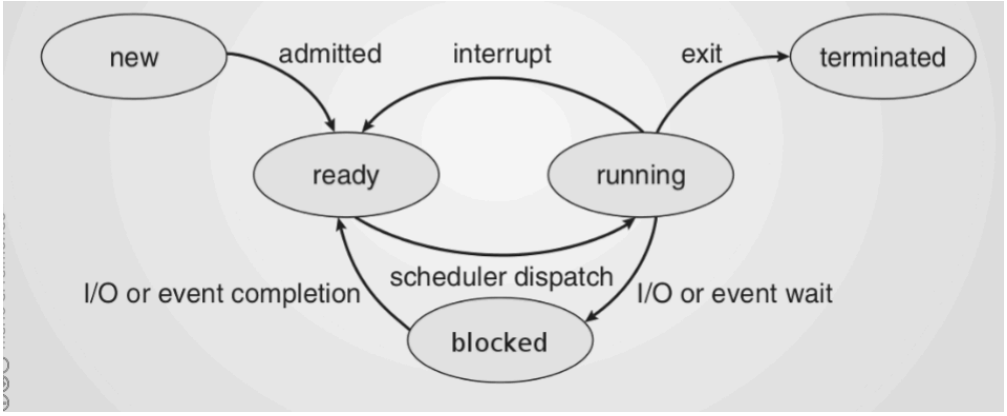
fork e exec usate in un certo modo possono riprodurre il funzionamento di CreateProcess

Terminazione del processo: esistono diverse modi per terminare un processo:

- uscita normale: exit o ExitProcess, è il processo stesso che chiede al sistema operativo di voler terminare
- uscita su errore
- kill (UNIX) o TerminateProcess (windows) da parte di un altro processo (ad esempio quando chiudiamo una finestra con la X della GUI)

esiste soft-kill e hard-kill: il primo manda un segnale di terminazione, hard kill invece uccide "senza pietà" (ad esempio il termina attività del gestione attività)

Stato di un processo



1. **new**: il processo viene inizializzato
2. **ready**: il processo è pronto ad essere eseguito dalla CPU, viene messo in una *coda dei processi* è il sistema operativo a scegliere quale processo in fase di ready eseguire
3. **running**: Il processo che si trova nella coda dei processi viene scelto dal sistema operativo e viene eseguito
4. **blocked**: il processo in stato di running esegue una chiamata di input/output molto lenta, il processo è bloccato perché sta aspettando la risposta di questa chiamata, quello che succede praticamente è che il sistema operativo lo tira fuori dalla coda dei processi a causa di questa cosa quest'ultimo entra nello stato di blocked, appena la chiamata di sistema rientra il processo diventa ready.
5. **terminated**: il processo ha finito, tutte le sue risorse sono state liberate.

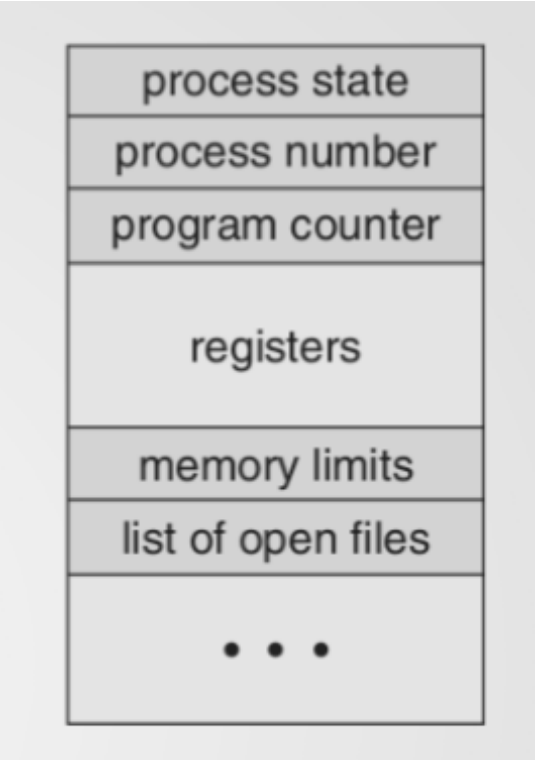
Se un processo non effettua operazioni di I/O rischia di monopolizzare la CPU. Per evitare che questo accada esiste la **prelazione**: un'operazione tramite la quale il Sistema Operativo sposta il processo dallo stato di *running* a quello di *ready*. Questo avviene quando il processo ha esaurito la quantità massima di tempo CPU che gli era stata assegnata, ed è innescato direttamente tramite un *interrupt*.

quando cambia il processo in esecuzione parliamo di cambio di contesto di esecuzione, questo viene fatto dall'operazione di **dispatch** gestita dal dispacher che è quell'unità

del SO che gestisce il controllo della CPU da parte dei processi consegnandolo al processo che deve essere eseguito, lo scheduler invece è l'algoritmo che si occupa di scegliere quale processo deve essere eseguito

Tabella dei processi

All'interno della tabella dei processi troviamo vari PCB, in un PCB come dicevamo prima troviamo le seguenti informazioni



Queste informazioni vengono usate in diversi modi, sopratutto nella gestione degli interrupt. Di seguito la gestione degli interrupt per il passaggio da un processo ad un altro:

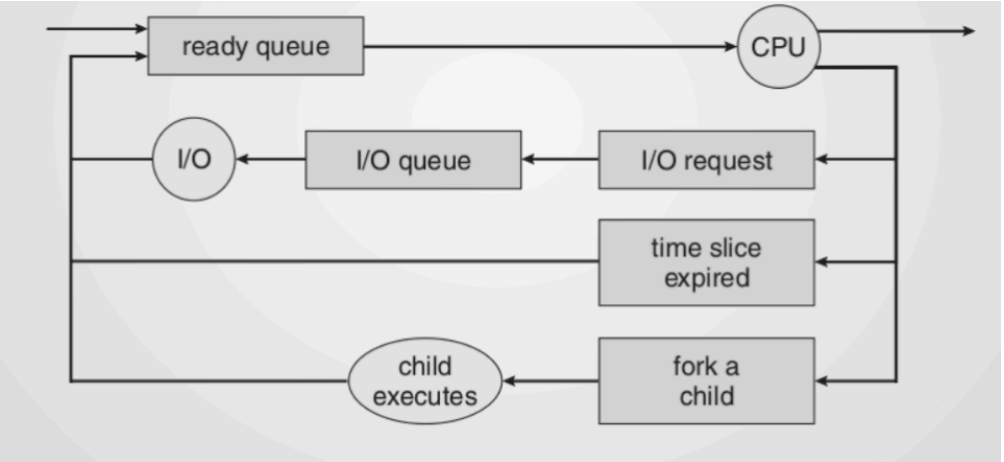
- salvataggio nello stack del PC e del PSW nello stack attuale;
- caricamento dal vettore degli interrupt l'indirizzo della procedura associata;
- salvataggio registri e impostazione di un nuovo stack;
- esecuzione procedura di servizio per l'interrupt;
- interrogazione dello scheduler per sapere con quale processo proseguire;
- ripristino dal PCB dello stato di tale processo (registri, mappa memoria);
- ripresa nel processo corrente.

Tip

Il **PSW** (acronimo di **Program Status Word**, o a volte *Processor Status Word*) è un registro hardware fondamentale all'interno della CPU che descrive lo stato attuale del processore in un determinato istante.

Code e accodamento

Di seguito la gestione delle code all'interno della CPU durante l'esecuzione dei processi

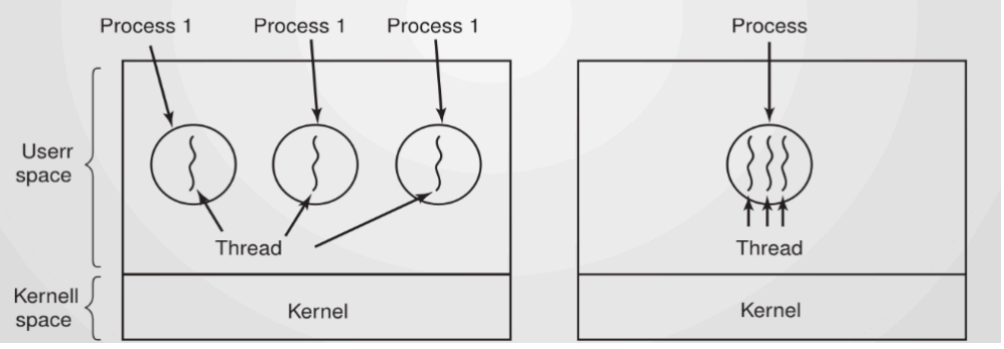


Vediamo che per ogni caso abbiamo abbiamo una coda:

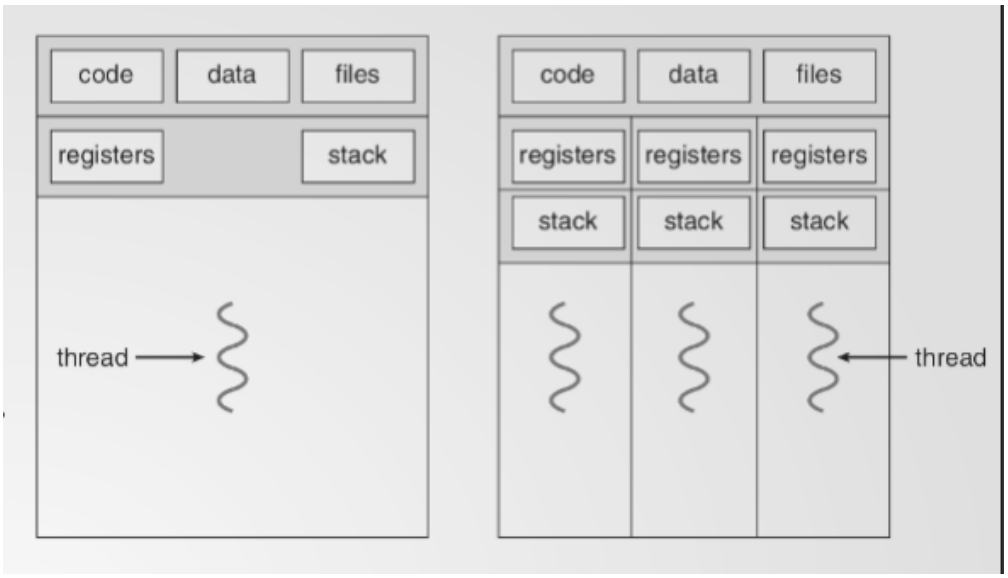
- I/O queue/request: la coda di tutti i processi blocked a causa di operazoni di input/output
- time slice expired: la coda di tutti quei processi che "consumano" tutta la time slice
- fork a child: qua troviamo tutti i processi che hanno bisogno di tempo per forkarsi vediamo che tutto converge nella coda dei processi ready, da qui il sistema operativo pesca il processo da eseguire

Thread

All'interno di un processo si possono avere più flussi di esecuzione, questi flussi sono i thread (lightweight processes).
Ad esempio se ho il processo di brave browser ci sarà un thread che gestisce la visualizzazione delle pagine, un altro che gestisce l'animazione della barra di ricerca.
Questo implica che la nostra CPU è formata da più CPU virtuali. Usare i thread ha dei benefici sopratutto durante lo stato "blocked" infatti al posto di bloccare tutto il processo si blocca solo lo specifico thread che fa l'operazione I/O.



Di seguito la differenza tra uso e non uso di thread



Come possiamo notare un Thread è caratterizzato da:

- Program Counter, registri, stack, stato
  e poi condivide tutto il resto con gli altri thread, nell'immagine vediamo che sia il codice, i dati e il file vengono condivisi. La condivisione dei dati crea dei bug se il programmatore non è molto abile nella programmazione concorrente

Vita del thread

Possiamo dire che i thread vivono dentro lo stesso processo, in pratica condivido lo spazio di indirizzamento del processo.

Tabella dei thread: Ovviamente esiste una tabella dei thread che viene usata dalla CPU per quando ne esegue uno piuttosto che un'altro (questa tabella contiene una specie di PCB), quando passo da un thread ad un altro sto parlando di cambio del contesto di esecuzione, questa operazione è più efficace nei thread piuttosto che nei processi, questo perché lo spazio di indirizzamento è in comune.
Stati: sono praticamente uguali a quelli dei processi (Running, Ready, Blocked)
Operazioni: sui thread possiamo fare diverse operazioni necessarie per gestirli

- *thread_create*: un thread ne crea un altro;
- *thread_exit*: il thread chiamante termina;
- *thread_join*: un thread si sincronizza con la fine di un altro thread, questa cosa è utile in alcuni casi, lo vedremo al lab.
- *thread_yield*: il thread chiamante rilascia volontariamente la CPU.